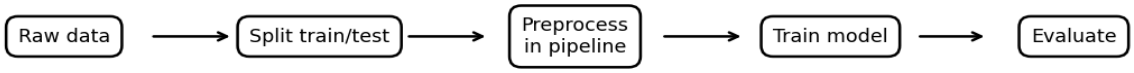
Day 32: Feature Engineering and Scikit-learn Pipelines

A practical guide to missing values, scaling, encoding, ColumnTransformer, Pipeline, and leakage prevention

Learning goal: create a consistent, reusable machine learning workflow and evaluate it without allowing test data to influence training.



Important: the accompanying dataset is tiny and synthetic. It teaches the workflow and does not support real-world conclusions.

1. What is feature engineering?

Feature engineering transforms raw columns into useful model inputs. Examples include filling missing values, encoding categories, scaling numeric values, extracting date components, creating ratios, or applying domain knowledge. A feature must be available at the time a prediction is made. A feature derived from the answer or future information can create leakage.

For example, a department label such as “Computer Science” is meaningful to a person, but many estimators need numeric features. OneHotEncoder can turn categories into indicator columns.

2. Missing values and imputation

Missing data can appear as NaN, blank cells, or unknown values. SimpleImputer learns a replacement strategy from the data passed to fit. **Mean** uses the average and can be affected by outliers. **Median** uses the middle value and is often more robust. **Most frequent** fills with the most common value, often useful for categories. **Constant** uses a chosen placeholder such as “Unknown”.

Do not fit an imputer on the entire dataset before splitting. The test data would influence training preprocessing. Keeping the imputer inside a Pipeline lets cross-validation fit it separately within each training fold.

3. Scaling numeric features

StandardScaler standardizes each feature using z = (x – mean) / standard deviation. **MinMaxScaler** maps values into a chosen range, often 0 to 1, using the training minimum and maximum. The learned values are then applied to validation, test, and future data.

Scaling is often important for distance-based or gradient-based methods such as K-nearest neighbors, support vector machines, and many linear models. Tree-based models such as Random Forests generally do not require scaling. Scaling does not guarantee a normal distribution.

4. Encoding categorical features

OneHotEncoder creates binary columns to represent each category. This avoids suggesting that unordered categories have a numeric ranking.

Department	CS	IT	SE
CS	1	0	0
IT	0	1	0
SE	0	0	1

The setting **handle_unknown='ignore'** helps the fitted encoder handle a category that was not seen during training without raising an error. The new category will not activate any of the known category columns.

5. Pipeline

Pipeline chains transformations and an estimator. During fit, each step learns its settings from the training data and passes transformed output to the next step. During prediction, the same fitted transformations are applied automatically. This reduces the risk that training and inference use different preprocessing.

6. ColumnTransformer

ColumnTransformer applies different transformations to different column groups. A common design is numeric columns → median imputation → scaling, and categorical columns → most-frequent imputation → one-hot encoding. The outputs are combined into the feature matrix used by the estimator.

7. Safe workflow from start to finish

1) Separate X (features) and y (target). 2) Split train and test. 3) Define numeric and categorical pipelines. 4) Combine them with ColumnTransformer. 5) Add preprocessing and model to one Pipeline. 6) Run cross-validation on training data only. 7) Fit the pipeline on the training partition. 8) Evaluate on held-out test data. 9) Use the fitted pipeline for new examples.

8. Core code pattern

```
numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])
categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])
preprocessor = ColumnTransformer(transformers=[
    ("numeric", numeric_pipeline, numeric_features),
    ("categorical", categorical_pipeline, categorical_features),
])
model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(n_estimators=100, random_state=42)),
])
```

The complete runnable code is in **Day-32-Python-Practice.py**. It adds missing values to a toy dataset, splits the data, performs cross-validation on the training partition, fits the pipeline, evaluates held-out examples, and predicts one new example.

9. Data leakage: the critical rule

Data leakage happens when a model gets information it should not have during training or evaluation. Fitting an imputer or scaler on all rows before splitting is one example. Using target-derived columns or information from the future is another. Leakage can make validation results look much better than performance in real use.

Safer approach: split first, then fit transformations through the pipeline during training and cross-validation. Keep the test set out of feature selection and hyperparameter tuning. Pipelines help prevent preprocessing leakage but cannot automatically identify every form of target or time leakage.

10. Common mistakes and fixes

Text column error: encode categories. **NaN error:** add imputation. **Unseen category:** consider handle_unknown='ignore'. **Preprocessing differs between training and prediction:** reuse the fitted pipeline. **Suspiciously high score:** investigate

leakage, duplicates, target-derived columns, and whether the split matches the intended use.

11. Exercises with solutions

Q1. Why fit imputation on training data only? To keep test information from influencing training preprocessing.

Q2. Which transformer is a common choice for an unordered department column? OneHotEncoder.

Q3. What does ColumnTransformer do? It applies different preprocessing to specified groups of columns.

Q4. Why use Pipeline? It keeps preprocessing and modeling steps together, supports leakage-aware cross-validation, and makes prediction preprocessing consistent.

Q5. What does handle_unknown='ignore' do? It prevents errors when an unseen category appears at inference time.

Q6. Does scaling always improve Random Forest? No. Tree-based models generally do not depend on feature scale.

12. Revision checklist

■ Explain feature engineering. ■ Compare mean, median, most-frequent, and constant imputation. ■ Explain StandardScaler and MinMaxScaler. ■ Describe one-hot encoding. ■ Build Pipeline and ColumnTransformer. ■ Split before fitting preprocessing. ■ Keep cross-validation and test evaluation leakage-aware. ■ Explain why scores from tiny synthetic data are unstable.

13. Final takeaway

A dependable machine learning solution is a workflow, not just an algorithm. Separating data splitting, preprocessing, training, validation, and inference makes the code easier to maintain and the evaluation more trustworthy. Once the workflow is clear, practise it on a suitable real dataset.

Suggested documentation: scikit-learn User Guide topics covering Pipeline, ColumnTransformer, imputation, preprocessing, and model selection.